```
    case Result of
    atomic -> {ok, Body} = webapp_dtl:render([{the_list,
SendList}]);
    _ -> {ok, Body} = webapp_dtl:render([{the_list, []}])
    end,
    % webmachine expects a tuple
    {Body, ReqData, Context}.
```

The resource module needs at least two functions by default—*init* and *to_html*. The *init* function is used to specify the context needed for this resource. Webmachine passes it, as is, to various functions.

The default name of the function for the GET request is *to_html*. In case the key field is empty or not defined, the function does nothing; otherwise, it inserts the key/value pair in a table. It then queries the table for all records that have the desired value and displays the result in a template.

The *templates/webapp.dtl* template is similar to the one explained in the previous article:

```
<html>
  <body>
    <form method="GET">
      <p>Key: <input type="text" name="key">
      Value: <input type="text" name="value"></p>
      <p> Search for Value" <input type="text" name="search_
for"></p>
      <input type="submit">
    </form>
    {% for table,key,value in the_list %}
      <p> {{ key }}   Value {{ value }}  </p>
    {% endfor %}
  </body>
</html>
```

For completeness, the code in src/db.erl for storing and querying data from mnesia is as follows:

```
-module(db).
-export([init/0,insert_key/2,read_data/1,match_value/1]).
-record(store,{key,value}).
init() ->
    mnesia:create_table(store,
                    [{attributes,record_info(fields,
store)}]).
insert_key(K,V) ->
    F=fun() -> mnesia:write(#store{key=K,value=V})
      end,
    mnesia:transaction(F).
match_value(Val) ->
    Pattern = #store{_ = '_', value=Val},
    F = fun() ->
      mnesia:match_object(Pattern)
      end,
    mnesia:transaction(F).
```

Finally, *mnesia* needs to be started. Add the following lines at the beginning of the *start* function in *src/webmachine_demo.erl:*

```
    application:set_env(mnesia,dir,"db"),
    mnesia:create_schema([node()]),
    ensure_started(mnesia),
    db:init(),
```

Compile and run *start.sh*. Now browse *http://localhost:8000/key* and explore the form.

Getting used to Erlang is not difficult. Functional programming is fun. END

References

[1] *https://github.com/basho/webmachine/wiki*
[2] *http://www.erlang.org/doc/apps/mnesia/Mnesia_chap2.html*
[3] *http://en.wikiversity.org/wiki/Web_Development_with_Webmachine_for_Erlang*

By: Anil Seth

The author has earned the right to do what interests him. You can find him online at *http://sethanil.com, http://sethanil.blogspot.com,* and reach him via email at *anil@sethanil.com*